BAB 6 | Intro to Client-Side (React)

Pada bab-bab sebelumnya, kita mempelajari bagaimana JavaScript memanipulasi DOM secara langsung. Pada bab ini, kita akan belajar **React**, sebuah *library* JavaScript untuk membangun antarmuka pengguna (UI) secara lebih terstruktur, dengan cara memecah tampilan menjadi komponen-komponen kecil yang bisa digunakan berulang.

Sebagai studi kasus yang akan kita bangun sepanjang bab ini, kita akan membuat sebuah **website profil sederhana** yang terdiri dari Navbar, Hero, Galeri Foto, dan Music Player.

6.1 Pengenalan React

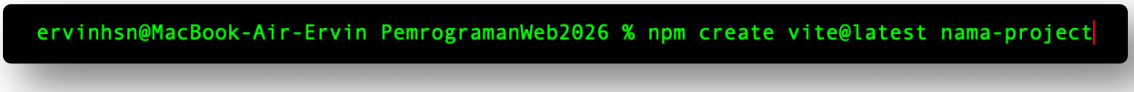
React adalah *library* JavaScript untuk membangun UI dengan pendekatan **berbasis komponen** (*component-based*). Alih-alih menulis satu file HTML besar berisi seluruh tampilan, kita memecahnya menjadi bagian-bagian kecil dan mandiri (Navbar, Hero, Footer, dsb), lalu menggabungkannya kembali.

6.1.1 SPA vs MPA

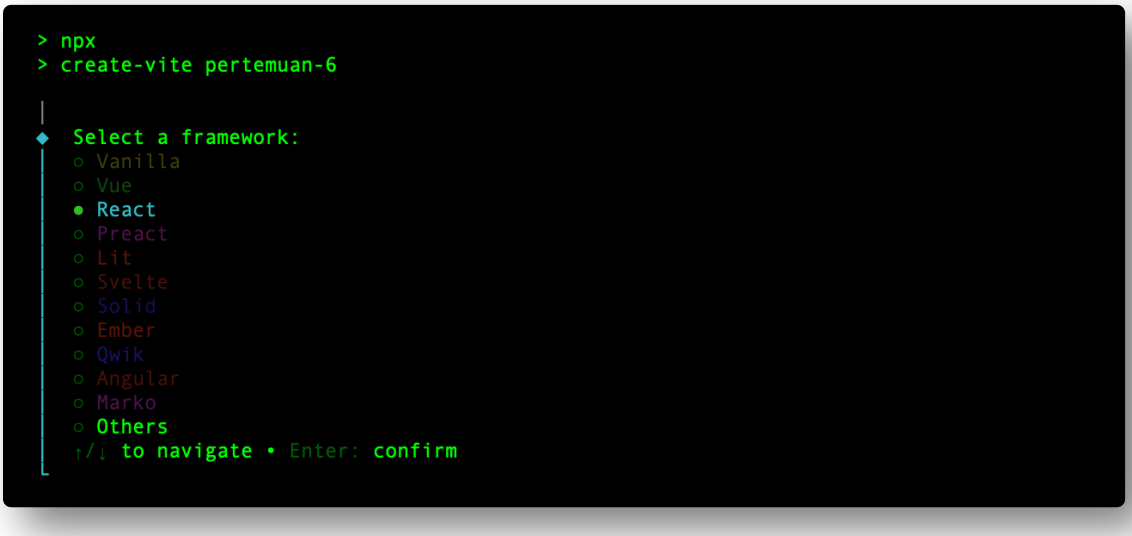
Konsep	Multi Page Application (MPA)	Single Page Application (SPA)
Navigasi halaman	Setiap navigasi memuat ulang (reload) halaman dari server	Hanya satu halaman HTML dimuat, konten diperbarui secara dinamis oleh JavaScript
Contoh	Website tradisional (banyak file .html terpisah)	Gmail, Instagram, aplikasi berbasis React pada umumnya

6.2 Persiapan Environment

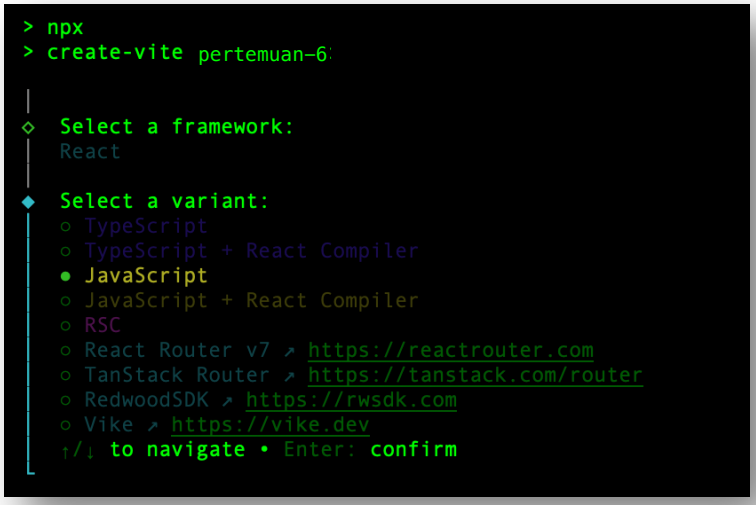
1. Buka terminal folder kalian lalu ketik "**npm create vite@latest nama-project**"



2. Pilih framework “**React**”



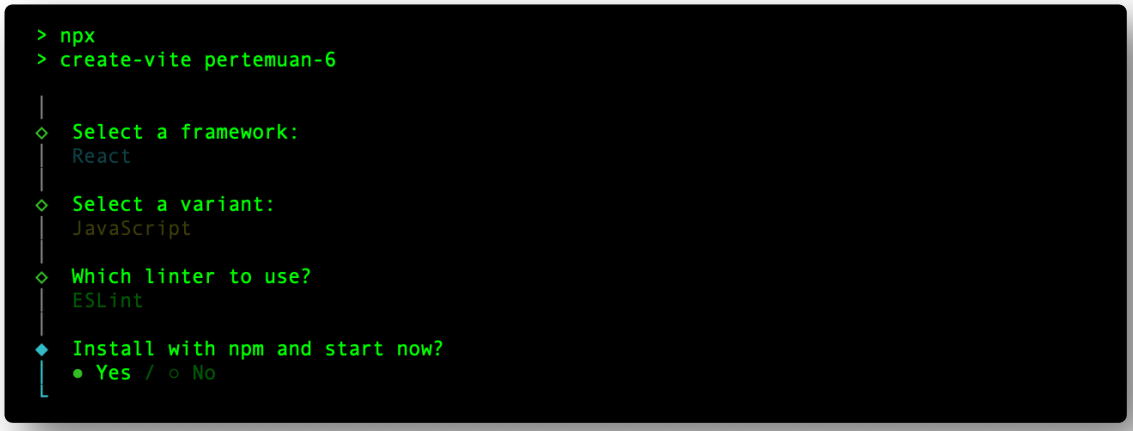
3. Pilih Varian “**JavaScript**”



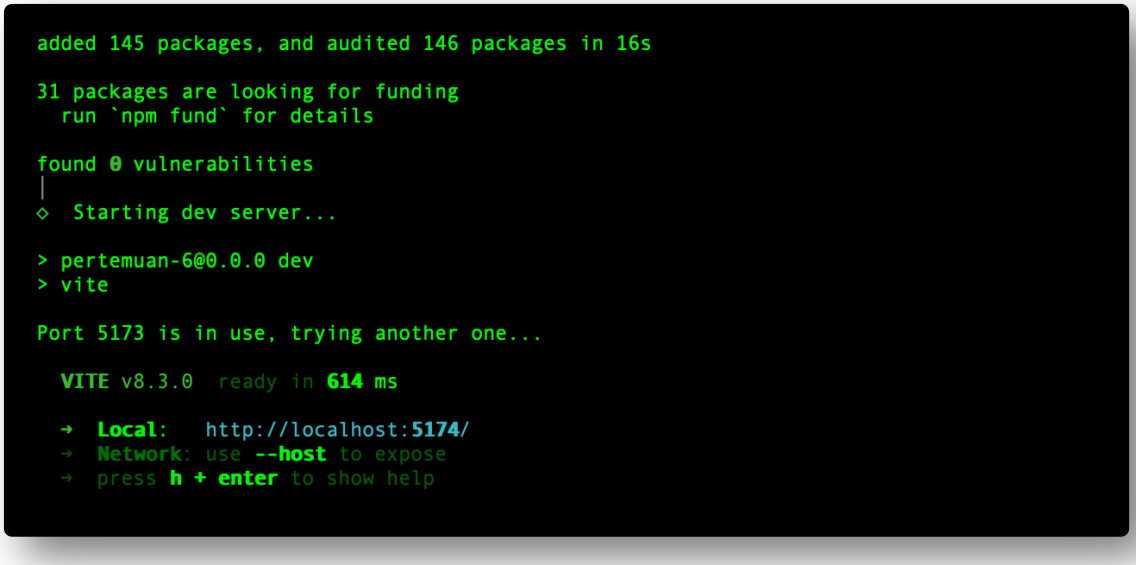
4. Pilih linter “**ESLint**”



5. Pilih “**Yes**” untuk menginstall npm



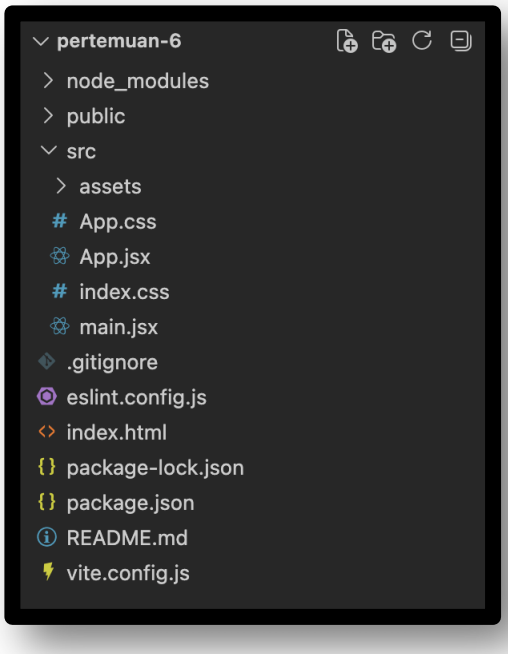
6. Setelah tampilannya seperti berikut anda siap memulai project anda



7. Masuk ke dalam folder project yang baru saja dibuat dengan mengetikkan perintah cd nama-project di terminal, lalu ketik code . untuk membukanya langsung di Visual Studio Code

6.2.1 Struktur Folder Project & Alur "Siapa Memanggil Siapa"

Ini bagian yang sering membingungkan di awal: file mana yang sebenarnya jadi titik masuk aplikasi, dan bagaimana komponen kita bisa muncul di browser?



Alurnya berjalan seperti rantai berikut:

1. **index.html** : satu-satunya file HTML dalam SPA. Isinya hanya kerangka kosong dengan satu ** ** sebagai "wadah":

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>modul6</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

2. **main.jsx** : inilah **titik masuk sebenarnya** dari aplikasi React. File inilah yang "menyuntikkan" komponen **App** ke dalam ** ** di atas:

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

```

Jadi urutannya: **index.html** menyiapkan wadah kosong → **main.jsx** mengambil wadah itu **(getElementById("root"))** → lalu merender komponen **<App />** ke dalamnya. Kita **hampir tidak pernah menyentuh file ini lagi** setelah setup awal, semua pekerjaan kita selanjutnya ada di **App.jsx** dan komponen-komponen turunannya.

3. **App.jsx** : komponen utama yang akan memanggil komponen-komponen lain (Navbar, Hero, dst). Ini yang akan terus kita kembangkan sepanjang bab ini.

```jsx
import { useState } from 'react'
import heroImg from './assets/hero.png'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import './App.css'

function App() {
  return (
    <div className="calculator">
    </div>
  );
}

export default App;
```

6.3 Anatomi Sebuah Komponen React

Komponen React, dalam bentuk paling dasar, hanyalah **sebuah fungsi JavaScript biasa** yang mengembalikan (**return**) sesuatu yang terlihat seperti HTML (disebut JSX).



```jsx
function Hero() {
  return (
    <section className="hero">
      <h1>Halo, Saya Ervin</h1>
      <p>Mahasiswa Sistem Informasi</p>
    </section>
  );
}

export default Hero;
```

Mari bedah kenapa setiap bagian ditulis seperti itu:

6.3.1 Kenapa harus function?

Karena komponen React memang didefinisikan sebagai fungsi. Nama fungsi ini (**Hero**) yang nantinya akan dipakai untuk memanggil komponen tersebut di tempat lain. React akan menjalankan fungsi ini setiap kali komponen perlu ditampilkan atau diperbarui.

6.3.2 Kenapa harus return?

Karena tugas komponen adalah "menjawab pertanyaan": *tampilan apa yang harus dirender?* Nilai yang di-*return* itulah JSX yang akan diubah React menjadi elemen HTML sungguhan di browser. Tanpa **return**, fungsi tidak menghasilkan apa-apa, sehingga tidak ada yang bisa ditampilkan.

6.3.3 Kenapa nama komponen Hero (huruf kapital di awal), bukan hero?

Ini bukan sekadar gaya penulisan, React memakai aturan ini untuk **membedakan komponen custom dari elemen HTML bawaan**. Ketika React membaca **<hero />** (huruf

kecil), ia akan mengiranya sebagai tag HTML biasa (yang tidak dikenali). Ketika membaca **<Hero />** (huruf kapital), React tahu itu adalah komponen buatan kita.

6.4 export dan import : Kenapa Wajib Ada Keduanya?

Setiap file komponen di React berdiri sendiri sebagai sebuah **modul**. Modul tidak otomatis bisa "dilihat" oleh file lain, kita harus secara eksplisit **mengekspor** (membuka akses) dari file asal, dan **mengimpor** (mengambil akses) dari file tujuan.

export default di akhir file **Hero.jsx** artinya: "inilah hal utama yang boleh diambil dari file ini oleh file lain."

```
10  export default Hero;
```

Kata **default** berarti hanya boleh ada **satu** export default per file, dan saat diimpor, kita bebas memberi nama apa saja (walau sebaiknya tetap konsisten dengan nama aslinya agar tidak membingungkan).

Di **App.jsx**, kita **wajib mengimpor** komponen tersebut dulu sebelum bisa memakainya:

```
6   import Hero from "./components/Hero.jsx";
```

Kalau baris import ini dihapus, kode akan error meskipun **Hero.jsx** sudah ada di folder — karena **App.jsx** tidak tahu-menahu soal file lain kecuali diberi tahu lewat **import**.

6.4.1 Perbedaan "./" dan "../" pada Path Import

Path pada **import** bersifat **relatif terhadap lokasi file yang sedang menulis import itu sendiri**, persis seperti konsep *relative path* pada **href** di Bab 1.4.2.

Path	Artinya
"./Hero.jsx"	Cari file **Hero.jsx** di folder yang sama dengan file ini
"../Hero.jsx"	Naik satu folder ke atas dulu, baru cari **Hero.jsx** di sana

"./components/Hero.jsx"	Cari folder **components** di folder yang sama, lalu ambil **Hero.jsx** di dalamnya

Contoh pada struktur folder kita: karena **App.jsx** berada di **src/** dan **Hero.jsx** berada di **src/components/**, maka import dari **App.jsx** ditulis:

```
6    import Hero from "./components/Hero.jsx";
```

Tapi jika ada komponen lain di dalam **src/components/** yang ingin mengimpor komponen lain **yang sejajar** (satu folder yang sama), cukup pakai **"./"**:

6.5 Memanggil Komponen

Setelah diimpor, komponen dipanggil di dalam JSX **layaknya tag HTML**, menggunakan format *self-closing tag* **<NamaKomponen />**:

```
1   import { useState } from 'react'
2   import heroImg from './assets/hero.png'
3   import reactLogo from './assets/react.svg'
4   import viteLogo from './assets/vite.svg'
5   import Hero from "./components/Hero.jsx";
6   import './App.css'
7
8   function App() {
9     return (
10      <div className="app">
11        <Hero/>
12      </div>
13    );
14  }
15
16  export default App;
```

Tag **<Navbar />** ini pada dasarnya sama saja dengan menjalankan fungsi **Navbar()** dan menaruh hasil **return**-nya di posisi tersebut. React akan "menyuntikkan" apa pun yang di-*return* oleh fungsi **Navbar** ke titik itu.

> ***Catatan:*** *boleh juga ditulis* ***<Navbar></Navbar>*** *(buka-tutup) jika di dalamnya ingin diisi konten lain, tapi karena komponen kita saat ini tidak menerima isi tambahan, cukup ditulis singkat* ***<Navbar />.***

6.6 Menyusun Studi Kasus: Website Profil

Sekarang kita terapkan seluruh konsep di atas untuk membangun komponen-komponen website profil kita satu per satu.

1. File "**src/components/Navbar.jsx**"

```jsx
function Navbar() {
    return (
        <nav className="navbar">
            <h2>MyPortfolio</h2>
            <ul>
                <li>Home</li>
                <li>Gallery</li>
                <li>Music</li>
                <li>Contact</li>
            </ul>
        </nav>
    );
}

export default Navbar;
```

2. File “**src/components/Hero.jsx**”

```jsx
function Hero() {
  return (
    <section className="hero">
      <h1>Halo, Saya Ervin</h1>
      <p>Mahasiswa Sistem Informasi</p>
    </section>
  );
}

export default Hero;
```

3. File “**src/components/Footer.jsx**”

```jsx
function Footer() {
    return (
        <footer>
            <p>&copy; 2026 MyPortfolio. All rights reserved.</p>
        </footer>
    );
}

export default Footer;
```

4. File “**src/components/App.jsx**”, di sinilah semua komponen tadi dikumpulkan dan dipanggil

```jsx
import Hero from "./components/Hero.jsx";
import Navbar from './components/Navbar.jsx';
import Footer from './components/Footer.jsx';
import './App.css'

function App() {
  return (
    <div className="app">
      <Navbar/>
      <Hero/>
      <Footer/>
    </div>
  );
}

export default App;

```

6.7 Kenapa Pakai const?

Di dalam komponen React, kita akan sering menyimpan data (misalnya daftar menu, daftar foto) ke dalam sebuah variabel sebelum ditampilkan. Untuk ini kita memakai **const**, bukan **let** atau **var**. Alasannya:

- **const** berarti variabel tersebut **tidak dimaksudkan untuk diganti nilainya** setelah didefinisikan. Karena setiap kali komponen dirender ulang, fungsi tersebut dijalankan dari awal, variabel dibuat ulang setiap kali, bukan "diubah" nilai sebelumnya.

- Memakai **const** sebagai kebiasaan membantu mencegah bug tak sengaja, karena compiler akan menegur jika kita mencoba menimpa nilai **const** di tempat yang salah.



```
function Navbar() {

    const menu = ["Home", "Gallery", "Music", "Contact"];

    return (
        <nav className="navbar">
            <h2>MyPortfolio</h2>
            <ul>
                {menu.map((item, index) => (
                    <li key={index}>{item}</li>
                ))}
            </ul>
        </nav>
    );

}

export default Navbar;
```

Nanti di Bab 7, *saat data* ***memang perlu berubah akibat interaksi pengguna*** *(misalnya menu aktif berpindah saat diklik), kita tidak memakai* ***let****, melainkan* ***useState****, fungsi khusus dari React yang akan dibahas di sana.*

6.8 Array.map() : Kenapa Dipakai untuk Menampilkan List?

Daripada menulis **<li>** satu per satu secara manual untuk tiap menu, kita gunakan **.map()** agar satu baris kode bisa menghasilkan banyak elemen sekaligus berdasarkan data array.

```jsx
function Navbar() {

    const menu = ["Home", "Gallery", "Music", "Contact"];

    return (
        <nav className="navbar">
            <h2>MyPortfolio</h2>
            <ul>
                {menu.map((item, index) => (
                    <li key={index}>{item}</li>
                ))}
            </ul>
        </nav>
    );

}

export default Navbar;
```

Cara membaca baris .map() di atas:

- **menu.map(...)** → jalankan sesuatu untuk **setiap item** di dalam array **menu**.
- **(item, index) => (...)** → **item** adalah nilai tiap elemen **("Home"**, **"Gallery"**, dst), **index** adalah nomor urutnya (0, 1, 2, ...).
- Hasil dari **.map()** adalah **array baru berisi elemen JSX**, inilah kenapa hasilnya bisa langsung ditaruh di dalam {} di antara **
** dan **
**.
- **key={index} wajib ada** setiap kali me-render list dengan **.map()**, agar React bisa melacak elemen mana yang berubah secara efisien saat tampilan diperbarui.

6.9 Event Handling

Penanganan event di React mirip HTML/JavaScript biasa (lihat kembali cara kerja **addEventListener** di Bab 5), namun ditulis sebagai atribut JSX dalam format camelCase, dan nilainya berupa **referensi fungsi**, bukan string.

HTML/JS Manual	React (JSX)
<button onclick="handleClick()">	**<button onClick={handleClick}>**
Fungsi dipanggil sebagai teks/string	Fungsi diberikan sebagai referensi (tanpa tanda kurung ())

console.log() bukan bagian dari React, ini fungsi bawaan JavaScript untuk mencetak nilai ke **Console** pada DevTools browser (klik kanan → Inspect → tab Console). Fungsinya murni untuk **membantu kita men-debug**, memastikan kode kita "menangkap" nilai yang benar, tanpa harus menampilkannya ke pengguna di layer, jadi gunakan **console.log()** untuk mengecek apakah handling berfungsi dengan baik.

.

6.10 Menambahkan Gambar

React (dengan Vite) menyediakan dua cara menaruh gambar:

1. **Import langsung** (untuk gambar yang jadi bagian dari kode, misalnya foto galeri)

```
1  import ervinProfile from "../assets/gallery/ervinProfile.jpg"
2
3  function Gallery() {
4      return <img src={ervinProfile} alt="Wisuda" />;
5  }
```

Sama seperti komponen, gambar juga **harus diimpor dulu** sebelum dipakai — bedanya di sini yang diimpor bukan fungsi komponen, melainkan sebuah **path/URL** menuju file gambar hasil olahan Vite.

2. **Folder public/** (untuk aset statis, akses langsung tanpa import)

```
function Hero() {
  return <img src="/images/ervinProfile.jpg" alt="Foto Profil" />;
}

export default Hero;
```

Cara	Kapan digunakan
Import dari **src/assets/**	Gambar yang jadi bagian dari kode, akan dioptimasi Vite saat build
Folder **public/**	Aset yang cukup diakses lewat path tetap, tidak perlu diproses

6.10.1 Studi Kasus: Galeri Foto

```jsx
import ervinProfile from "../assets/gallery/ervinProfile.jpg"
import foto2 from "../assets/gallery/Hero.png"

function Gallery() {
    const galeriFoto = [
        { id: 1, src: ervinProfile, judul: "Wisuda" },
        { id: 2, src: foto2, judul: "Study Tour" },
    ];
    return (
        <section className="gallery">
            <h2>Galeri</h2>
            <div className="gallery-grid">
                {galeriFoto.map((foto) => (
                    <figure key={foto.id}>
                        <img src={foto.src} alt={foto.judul} />
                        <figcaption>{foto.judul}</figcaption>
                    </figure>
                ))}
            </div>
        </section>
    );
}
export default Gallery;

```

Perhatikan di sini **key={foto.id}**, kita pakai **id** dari data (bukan index array) karena setiap foto sudah punya id unik sendiri. Ini praktik yang lebih disarankan dibanding memakai index, terutama jika urutan data bisa berubah nantinya.

6.11 Menyatukan Semua Komponen di File “src/App.jsx”

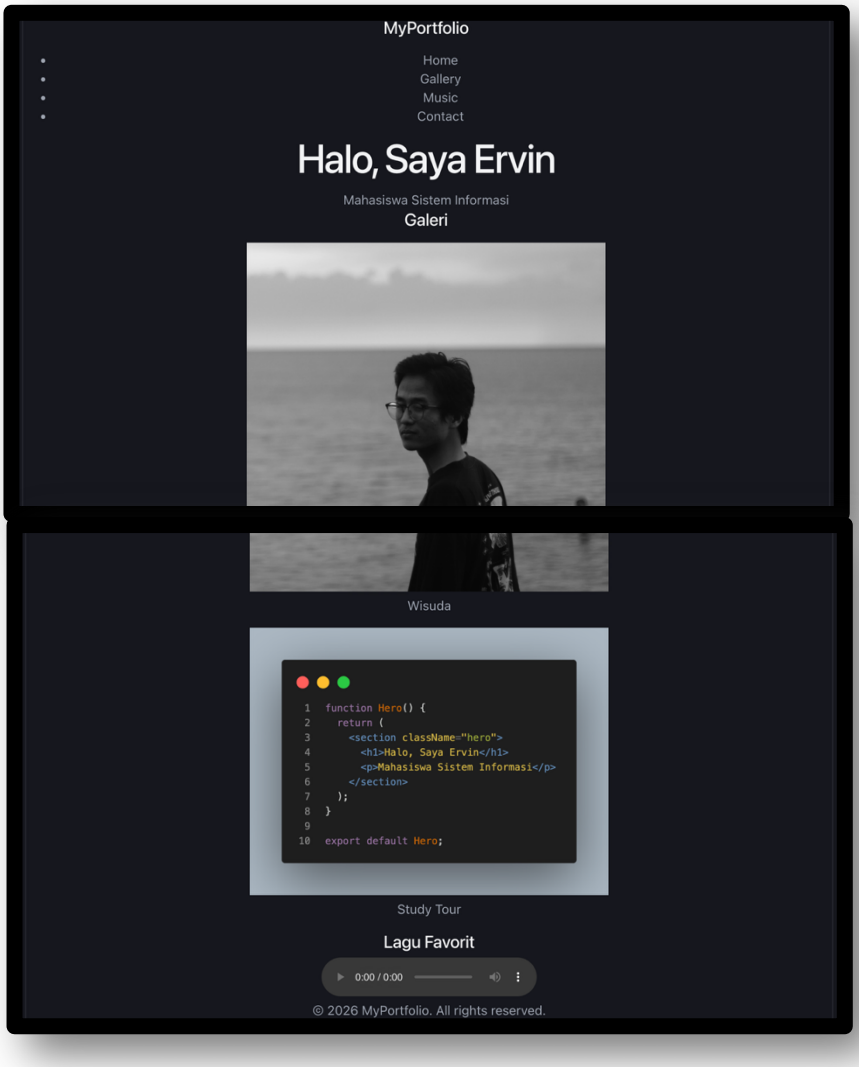
```jsx
import Hero from "./components/Hero.jsx";
import Navbar from './components/Navbar.jsx';
import Footer from './components/Footer.jsx';
import Gallery from "./components/Gallery.jsx";
import MusicPlayer from "./components/MusicPlayer.jsx";
import './App.css'

function App() {
  return (
    <div className="app">
      <Navbar/>
      <Hero/>
      <Gallery/>
      <MusicPlayer/>
      <Footer/>
    </div>
  );
}

export default App;

```

6.12 Hasil



Catatan penutup bab: *Website ini sudah tampil utuh — navbar, foto, dan pemutar musik — dan tombol menu sudah bisa diklik (tercatat di console). Namun menu yang diklik belum bisa "menandai dirinya aktif" secara visual, dan mengirim data (seperti daftar foto) dari satu komponen ke komponen lain masih dilakukan dengan menuliskannya ulang di setiap file. Pada* ***Bab 7 – React Hooks (State & Props)****, kita akan belajar* ***useState*** *untuk menyimpan data yang bisa berubah, serta* ***props*** *untuk mengirim data antar komponen — sehingga website ini menjadi lebih dinamis dan terstruktur.*